Yamma: A Combinatorial Analysis

Cumulative Research Report

by

Savannah Chappus

Department of Mathematics and Statistics

Northern Arizona University

Supervisor
Dr. Dana Ernst

October 2025

Contents

Abstract

In this paper, we introduce the beginnings of combinatorial analysis of the three dimensional puzzle game Yamma. Not only does this research conduct a basic analysis of the game from a combinatorial game theory perspective, it furthers that basic analysis through the use of computing. This research is largely exploratory, intending only to perform the first research done (to our knowledge) on the Yamma game. We begin with an outline of our initial research questions, then proceed into the research, which is divided in three sections. First, we discuss the hands-on analysis of the gameplay, including our resulting conjectures and facts learned from this analysis, then we discuss the key role of code in furthering this research, and the computational approach to generating a gamegraph for the game. Finally, we discuss the current issues and open questions which we hope will continue to drive this research, as well as our thoughts for future work on the project.

1 | Background

The board game *Yamma*, first released on June 7, 2024 by **Very Special Games** is a 3-dimensional, spinning 4-in-a-row game that combines unique strategy with minimalist design. Due to the fact that this game was released less than 1 year ago (at the time of writing), there has been no formal research done on this game. We proceed then to introduce the game and some basic terminology.

The game is played on a triangular spinning board with triangular holes where bi-colored cubes are placed. The board and cubes can be seen in Figure 1.1

Figure 1.1: Example of the Yamma board game

The cubes are colored as demonstrated in Figure 1.2. This coloring ensures that when a cube is placed, two faces of one color and one face of the other are visible. Players take turns placing cubes in one of two manners which are described in the Original Research Proposal Chapter. The game is over when one of the players places a cube that yields a 4-in-a-row pattern; however, the 4-in-a-row can happen on any one of the three perspectives of the board. The Yamma board game is marked on each side of the triangle with a small mountain motif, which denote each perspective, and players must consider all three perspectives when checking for a win.

Prior to continuing, we would like to define some terminology which is used throughout the paper.

1.1 Terminology

- **General Game Theory Terms:**
  - *Play:* A move (or potential move) made by one of the players. In this case, a play is the act of placing a cube "corner down" in one of two locations: 1) a triangular hole in the board, or 2) on top of 3 supporting cubes.
  - *Game/Board State:* A moment in the game that is in between plays. If the players pause the game, that is a state of the game. In this paper, this will likely also be referred to as just a "state" [eg. "At this state..."].
  - *Win/Winning position:* A play which results in a 4-in-a-row pattern on the board when viewed from one of the three marked perspectives.
  - *Winning strategy:* In any state of the game, if one player can "force" a win for themselves by playing optimally, they are said to have the winning strategy. This depends on the current state

of the game.

  - *Gamegraph:* A structure in combinatorial game theory that describes every possible play of a game. In a game like Yamma, this graph is far too large to be generated by hand, and even too large to feasibly be viewed.

- **Yamma-Specific Terms:**
  - *Side length:* The number of playable spaces on each side of the triangular board. For the original Yamma board, the side length is 4.
  - *X-Partizan / X-Impartial:* Given side length X and gameplay style Partizan or Impartial, this label is unique for every board size for both versions of gameplay. See (TODO) for a sketch of the proof.
  - *Triangular/Truncated:* A **triangular** board has 1 space in each corner, whereas a **truncated** board has 2 spaces in each corner. This describes the general shape of a given Yamma board. We generalized our naming scheme to not need this reference, but there are several conjectures and facts that depend on the shape of the board. See Figure 3.1 for a visual reference.

In addition to the defined terms, we made some modifications to the game as it was described in the instructions. The reasoning for these changes is described primarily in Section 3.1, so here we only give the modifications without justification. First, we focused primarily on the impartial version of gameplay because we felt it was more interesting than the partizan version. Second, the majority of our research is done on the 3-Impartial game, which is scaled back from the original game, which has side length 4. These changes allowed us to more easily analyze the mechanics and structure of the game, and even scaling back in the way that we did was too large for some of our analysis, as can be seen in Sections 3.2 and 3.3.

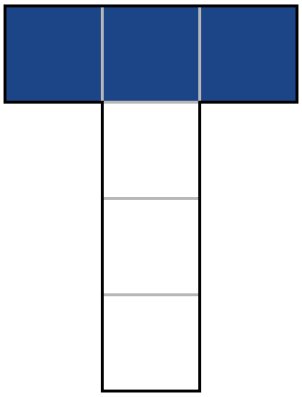

Figure 1.2: 2D layout of the coloring of Yamma cubes.

2 | Original Research Proposal

The recently-introduced game Yamma[1] is a two-player game in which players alternate stacking a single cube, each of which has 3 white sides and 3 blue sides. On each turn, a cube is placed "corner down" either in one of the open spaces of the game board or on top of three supporting cubes. After each placement, three sides of the cube are visible, two of which are one color and the remaining visible side is the other color. The board rotates to show multiple perspectives, so both players must focus on all sides as they play. The board that players play on is triangular with four spaces available along each side. See the image below for a visualization of the game.

There are two natural approaches to playing the game. In both versions of the game, a draw is possible.

- **Partizan:** This version of the game is the official one described in the instructions. For this version, one player is designated as white and the other blue. The objective is to be the first player to create a line of 4-in-a-row faces of their designated color when viewed from one of the three perspectives.
- **Impartial:** In this version of the game, players do not have a designated color. The objective is to be the first player to create a line of 4-in-a-row faces of either color when viewed from one of the three perspectives.

Loosely speaking, the goal of this project is to analyze both versions of the game. We propose to tackle some subset of the following problems.

Problem 1. Does the first or second player have a winning strategy in the partizan version of the game for the standard board size? Or will the game end in a draw if both players play optimally?

Problem 2. Does the first or second player have a winning strategy in the impartial version of the game for the standard board size? Or will the game end in a draw if both players play optimally?

Problem 3. Under impartial play, if the player that places the final cube in a configuration that results in a draw is designated as the loser, one can naturally assign a nim-value to each configuration of cubes. In this paradigm, can we determine the nim-value of the initial state or any intermediate configuration?

Problem 4. Up to symmetry (rotation and color), how many board configurations are possible (including intermediate gameplay)?

Problem 5. Up to symmetry (rotation and color), how many winning board configurations are possible?

Problem 6. Up to symmetry (rotation and color), how many winning board configurations that result in 5-in-a-row of a color are possible?

Problem 7. Up to symmetry (rotation and color), how many draw board configurations are possible?

Problem 8. One can naturally generalize the initial board size. Can we answer any of the previous problems for different board sizes?

[1]Link to the retail version of the game produced by **Very Special Games** is here.

3 | Research to date

Our research on Yamma has consisted of 3 general phases. The following sections detail the progress and results from each of these phases.

- **Experimentation and hands-on study.** This is where we played the game, observed the outcomes, devised conjectures, and attempted to discover winning strategies. This is also the phase in which we made decisions about the version of gameplay we would use for the majority of our research going forward.
- **Program Game Play.** Once we had devised a few conjectures, we were interested in potential avenues for utilizing code in our research. Here, we designed a digital version of the gameplay, complete with winning condition checks and a unique approach to "flattening" the 3-dimensional Yamma board.
- **Gamegraph Generation.** The third phase is where our research currently stands. Using the digital version of Yamma, we began designing a program that will generate the gamegraph for Yamma. While we have what we consider to be working code, there are some considerations regarding our approach. We will discuss this in later sections.

3.1 Experimentation and Hands-On Study

We started our research with a few quantitative questions about the general board layout. These are outlined below.

1. What are the side lengths of each board?
2. How many holes are in the board for each side length?
3. How many cubes fill the board for each side length?

The answer to Question 1 is given in Figure 3.1. Questions 2 and 3 are answered by Facts 11/12 and Conjectures 13/14 respectively. Table 3.1 also enumerates the total number of holes and cubes for side lengths where $0 \leq k \leq 9$.

Definition 9. The triangular numbers are represented by T_n, defined by $T_n = \binom{n+1}{2}$.

Definition 10. The tetrahedral numbers are represented by $Te_n = \binom{n+2}{3}$.

Figure 3.1: Side lengths and layouts of the first four board sizes. New positions (from previous board) are given in dark blue.

Fact 11. For a triangular board with side length $3k$ for $k \geq 1$, the total number of spaces (holes) is T_{3k}.

Fact 12. For a truncated board with side length $3k+1$ for $k \geq 1$, the total number of spaces (holes) is $T_{3k+3} - 3$.

Conjecture 13. For a triangular board with side length $3k$ for $k \geq 1$, the total number of cubes is Te_{3k}.

Conjecture 14. For a truncated board with side length $3k+1$ for $k \geq 1$, the total number of cubes is $Te_{3k+3} - 3(3k+3) - 2$.

Fact 15. Each cube only has 6 distinct placements. These can be seen in Figure 3.2.

After these initial questions and conjectures, we played a few different variations of the game to determine how best to proceed. We started by playing both partizan and impartial versions of all of the boards with side length ≤ 4. The smallest side length is 1, and this game is fairly simple and easy to analyze. After playing both 1-Partizan and 1-Impartial, we concluded the following.

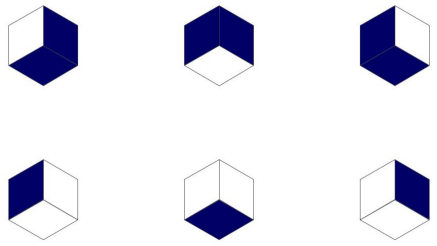

Figure 3.2: The 6 distinct placements of the Yamma cubes.

Fact 16. For 1-Partizan, player 2 has the winning strategy.

Fact 17. For 1-Impartial, player 2 has the winning strategy.

In both 1-Partizan and 1-Impartial, player 2's winning strategy is to just play optimally on their first turn. Due to the board's configuration and the coloring of the cubes, ≥ 1 face of each color will always be visible. This means player 2 only needs to match the color in one direction in order to win. We leave the proof of this to the reader.

At the same time, we also produced the following conjectures about generalizing the board game.

Conjecture 18. The sequence **A228888**[1] from OEIS enumerates the total number of cubes for triangular boards, with $k \geq 1$.

Table 3.1: Side length (s), total holes (h), and total cubes (c) for both triangular and truncated boards for $0 \leq k \leq 9$.

	Triangular			Truncated		
k	s	h	c	s	h	c
0	n/a	n/a	n/a	1	3	3
1	3	6	10	4	18	40
2	6	21	56	7	42	140
3	9	45	165	10	75	330
4	12	78	364	13	117	637
5	15	120	680	16	168	1088
6	18	171	1140	19	228	1710
7	21	231	1771	22	297	2530
8	24	300	2600	25	375	3575
9	27	378	3654	28	462	4872

3.2 Program Game Play

After playing the game and devising a few conjectures, we realized that computing the gamegraph by hand for even a smaller board like 1-Impartial would be inefficient, difficult, and prone to errors. The next step was to try to develop a program that could compute the gamegraph for any size board. The first issue that arose was how to "flatten" the 3D triangular pyramid-like shape into something that we can write code for.

The main issue with this is that the majority of data representation in code doesn't account for the triangular structures that we were dealing with. Initially, we decided on a large matrix representation, which we hoped would allow us to simultaneously store the data and check for the winning conditions. The key to flattening the game board was to label each position on the board and also label the faces of each cube. The arbitrary labeling system we used can be seen in Figure 3.3. The original matrix storage system for the 3-Impartial game was 30×30 (3 faces * 10 total cubes), and matrix cells contained with either 'w' or 'b' depending on the color of the respective face. This enabled us to flatten the game board, but it didn't provide an intuitive method for visualizing the winning conditions of the board, and there was not a clear computational way to check for wins in this layout.

[1]The sequence can be accessed here.

From there we wondered if looking at each of the three perspectives individually would provide us with the missing information. In doing this, we wanted to make sure that whatever method we chose for viewing each perspective would lend itself well to visualizing the winning conditions. We designed three 3×3 perspective matrices, where, for each perspective, the cube whose position is furthest from the viewer is placed in the matrix at $(0, 0)$. See Figure 3.4 for a more detailed look at this process. The next question we needed to address was how to determine which cubes belonged to which locations in the matrices.

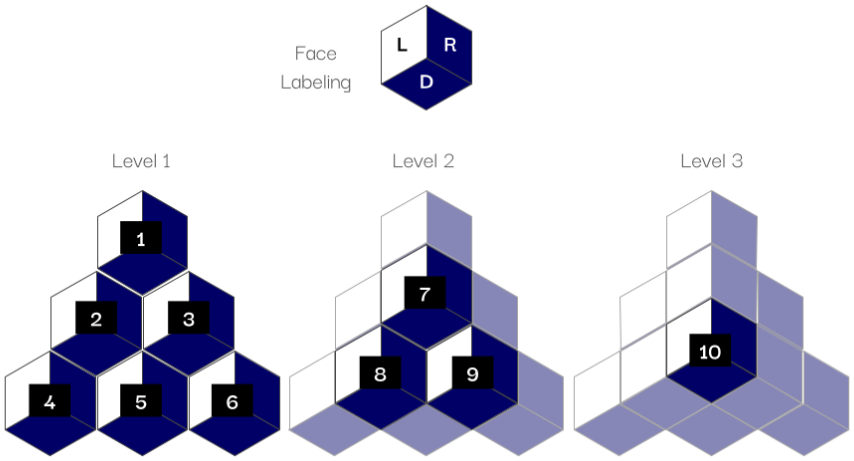

Figure 3.3: The arbitrary labeling scheme for the cube locations and faces.

Initially, we just manually determined which cubes belonged in which locations in the matrices, but we wanted to ensure that the program would be scalable, so we needed a way to generate the key matrices automatically. Through a little manipulation and some inspiration from the online game 2048[2], we were able to design a few small functions that are responsible for manipulating the three perspective matrices in order to get the cube labels into the correct location.

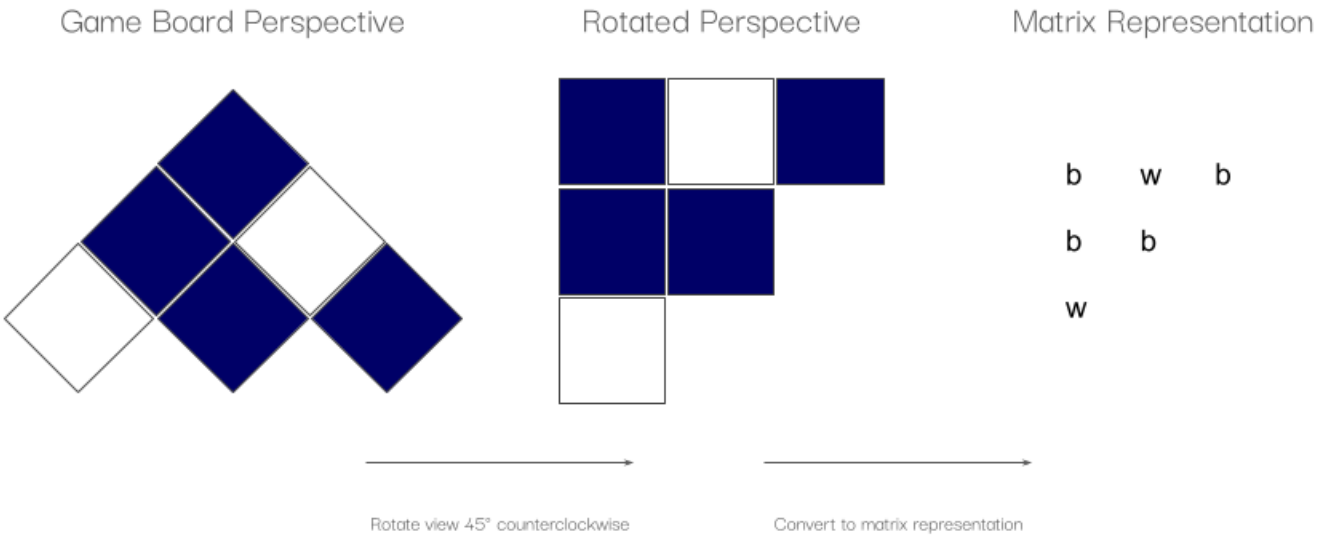

Figure 3.4: Process for translating game board perspective views into 2D matrices.

Using these three functions, we are able to generate key matrices for any size board. They key matrices allow us to determine which faces are updated when a cube is placed, and by condensing the game board into three 3×3 matrices representing each perspective, we are able to check the small perspective matrices for the winning condition in essentially the same manner that we do when we play the physical game. By ensuring that the cube furthest from the viewer is situated in the $(0, 0)$ position in the matrix, the winning

[2]Readers can play 2048 here.

$$\begin{matrix} * & b & b \\ b & w & \\ w & & \end{matrix}$$

$$\begin{matrix} 6,9,10 & 5,8 & 4 \\ 3,7 & 2 & \\ 1 & & \end{matrix}$$

(a) Top: Left face perspective matrix populated with an example game state. **Bottom:** Key matrix for cubes whose left face is visible in the this perspective view.

$$\begin{matrix} w & w & * \\ w & w & \\ w & & \end{matrix}$$

$$\begin{matrix} 1,7,10 & 3,9 & 6 \\ 2,8 & 5 & \\ 4 & & \end{matrix}$$

(b) Top: Down face perspective matrix populated with an example game state. **Bottom:** Key matrix for cubes whose down face is visible in the this perspective view.

$$\begin{matrix} w & w & w \\ b & w & \\ * & & \end{matrix}$$

$$\begin{matrix} 4,8,10 & 2,7 & 1 \\ 5,9 & 3 & \\ 6 & & \end{matrix}$$

(c) Top: Right face perspective matrix populated with an example game state. **Bottom:** Key matrix for cubes whose right face is visible in the this perspective view.

Figure 3.5: Example of the face and key matrices for all three perspectives.

condition is always row 1 or column 1 containing all the same value ('w' or 'b'). We do not need to consider the values along the diagonal because on the game board those cube faces are only adjacent by vertices, not edges, meaning that they do not meet the game's criteria for a winning position. An example of the 3 perspective matrices, populated with an example game board, as well as their corresponding key matrices, is given in Figure 3.5.

The game is played digitally through the terminal, and players take turns inputting their cube and its orientation in the command line. The play, given in the form "1 wwb" is interpreted as "place a cube in position 1 with the left face white, the down face white, and the right face blue" (recall the labeling of faces in Figure 3.3). The program takes this information and uses it to populate the 30-character string that holds the whole board layout, as well as the three perspective matrices. The program also has rudimentary checks for whether a cube is allowed to be placed, meaning it hasn't already been placed in that position and that it has three supporting cubes if it is being stacked. Finally, the program checks whether a winning condition has been met, and if so, terminates the program. The program for playing the game digitally is simple, and serves only as a means to the next stage: generating the gamegraph.

3.3 Gamegraph Generation

Once we had a way to play the game digitally, we proceed onto the actual goal, to construct a game graph for the Yamma game. Readers may recall the definition and general construction of gamegraphs as described in [1]. The structure of a gamegraph, a *digraph* (or directed graph) is a well-known data structure in computer science. These graphs are even easier to design and manipulate if they are *acyclic*, which the gamegraph for Yamma is, since you can never complete a turn without adding more information to the current board. The original gamegraph generation code is designed around several custom built classes in Python, which are responsible for storing the data for each node as well as the overall structure of the graph. The overall construction of these classes and all of the supporting functions is fairly straightforward, so the biggest issue we faced in this process is the massive size of these gamegraphs. The very simple 1-Impartial game (recall this has side-length 1 and 3 total cubes) has over 100 nodes in its gamegraph. For 3-Impartial, that number jumps to well over 200,000 nodes. The development of the program to generate a gamegraph went quickly, but generating the actual gamegraphs did not. In the beginning, generating just the first 3 levels of the gamegraph took on the order of days, even using the high-performance computing cluster we have at our disposal. In order to actually generate the gamegraph, we needed to make changes to speed up the process.

One of the immediate optimizations we made on the generation of the gamegraph was identifications for symmetry. The first identification we made was for the colors. We do not need to include both blue- and

white-dominant cube configurations in the gamegraph as long as we duplicate the total number of positions in the gamegraph (excepting the root node) at the end. This is because we can switch the colors on all cubes in all positions in the gamegraph and get the rest of the positions. The second identification we made was to make the first level of the gamegraph more manageable. Instead of having 6 opening positions, we determined that it is enough to consider placing the opening cube in one of two locations: (1) in a corner position or (2) in an edge position. This means that the first level of the gamegraph has 6 nodes rather than 36 nodes, which also makes the subsequent levels substantially smaller.

One of the major bottlenecks we encountered was that some nodes in the graph can be reached from multiple distinct parent positions. In generating the gamegraph, this meant that we were storing two nodes with exactly the same information and treating them as separate entities. We attempted to address this by checking for duplicate data as nodes were being generated, which we were unable to get functional under the time constraints of the project. As a quicker workaround, we generated all of the nodes in a particular level, then went back through and removed all of the duplicates after the fact. Initially, this seemed to speed up the generation of the gamegraph, but as the number of nodes grows in each level, the later levels took longer to generate all of the nodes and longer to remove the duplicates after the fact. This meant that what we thought would be a substantial speed up for the generation of the gamegraph would only speed it up for the lower levels and not for the entire gamegraph.

Due to the slow speed of the gamegraph generation process and our inability to devise a solution within the project's time frame, we were unable to generate the full gamegraph for the 3-Impartial game. However, we did generate some of the initial levels of the gamegraph, which can be seen in Figures 1-2 located in the Appendix. Despite not generating a full gamegraph for 3-Impartial, we did develop an outcome function which will be used on a complete gamegraph when one is available. The function serves to label each node with who has the winning strategy at any point in the game and is constructed from the bottom of the graph upward using the following rules:

1. All terminal positions are labeled with either:
   (a) 'L' if the previous move resulted in a winning position. This is a loss for the next player, hence 'L'.
   (b) 'D' if the previous move resulted in a draw.
2. All non-terminal positions are then labeled as such:
   (a) 'W' if there is at least one 'L' option in the option set.
   (b) 'L' if all options in the option set are 'W'.
   (c) 'D' if there are some 'D' options and some 'W' options in the option set.

3.4 Current Standing

This project is ongoing and, as such, we feel it is prudent to summarize the current state of the project as it stood at the time of writing. Here we enumerate the current issues with the code and the research as well as open questions. The issues presented here are without potential solutions. Please refer to the Future Work chapter for our ideas for solutions to some of these issues.

3.4.1 Issues

Below are a few of the most recently identified issues with this project. We have not included minor programmatic issues on this list as they are likely to be either fixed in a timely manner or ignored.

1. The generation of the gamegraph is inefficient (time and resources).

2. Handling duplicate children does not function as it should.

3.4.2 Open Questions

The following are the currently open research questions on this project. Some of these may require deep analysis and proof, and some may just be "simple" computational questions that we did not address within the scope of this paper. Note that many of the questions below also extend to larger board sizes.

1. How many total nodes does the gamegraph for 3-Impartial have?
2. Which player has the winning strategy in 3-Impartial?
3. What do winning strategies for 3-Impartial look like?
4. Are there other ways to represent the game in a 2-dimensional space that are better than the current representation?
5. How can we continue to optimize the code so that generating the gamegraphs for even larger boards is feasible?
6. What benefits could be reaped by parallelizing the code or utilizing a better language for large data structures like we are dealing with?
7. Is there a better way to store in-between states while generating the gamegraph so that the process can happen iteratively rather than all at once?

Note that all of the questions from our original research proposal went unanswered, (except for Problem 8), namely because we scaled the board down due to the immense size and complexity of the original board size. These questions are not duplicated here, but can be referenced in Chapter 2.

4 | Future Work

Finally, we would like to provide any additional ideas and goals we have as this project continues. We have divided this chapter into 3 sections as follows: 1) ideas for the problems or questions introduced in Section 3.4, 2) short-term goals that are likely to be completed soon, and 3) long-term goals that are likely to be accomplished further in the future.

4.1 Ideas

Any ideas we have that are related to the questions and problems from Section 3.4 are given here.

1. In our attempts to speed up generating the gamegraph, we realized that utilizing Python's `NetworkX` package may be more efficient than the current version, which was built from scratch using custom built Python classes. Using `NetworkX` may similarly have functionality to help deal with duplicate nodes.
2. Part way into the project, we realized that a vector-based approach for referencing the data and storing the board may be more "natural" when it comes to checking for winning conditions and manipulating data. We opted not to pursue this in this project, but we encourage any researcher who is willing and interested in attempting this type of representation for the data to do so and to reach out with their findings.
3. More recently, we very briefly attempted to parallelize some parts of the code, and can foresee significant performance improvements if this is done properly. This requires someone with adequate knowledge of parallelization and a significant amount of time to ensure that it is not only parallelized but also efficient otherwise (in order to reap the most benefits from the process). We attempted to use Python's `PoolProcessExecutor` to achieve a minimal amount of parallelization, but another tool may provide more benefit.
4. Currently, the gamegraph is generated on a level-by-level basis, and is exported using Python's `pickle` module, which is a built-in data serialization tool. While `pickle` is generally fast and reliable, we acknowledge that there may be better and more efficient tools out there. If the current data storage structure remains the same, exporting to `json` may provide structural and speed improvements, but it requires a fairly decent refactoring of the code. Conversely, the project may benefit from a shift to a different language that handles large-scale data structures better, like Java or C++.

4.2 Short-term Goals

The short-term goals are outlined below. Ideally, these should be completed within a 3 month time frame.

1. Shift the current graph storage structure to `NetworkX` and hopefully address the duplicate child issues.
2. Fix the issue in the game play logic that prevents us from analyzing 1-Impartial.
3. Pending completion of the previous goal, generate and analyze the gamegraph for 1-Impartial.

4. Complete the proofs for a few of the facts given in this paper.

4.3 Long-term Goals

The project's long-term goals are outlined below.

1. Optimize code so that we can generate gamegraphs for both 3-Impartial and 4-Impartial.
2. Conduct further analysis on what winning strategies look like for 3-Impartial.
3. Present research and findings in a small conference or colloquium setting.

The most up-to-date version of the code and supporting materials can be found on GitHub at the link here. Readers who are interested in contributing to this project should reach out to the author for the current status on the project, and then consult the GitHub for further information.

Appendix

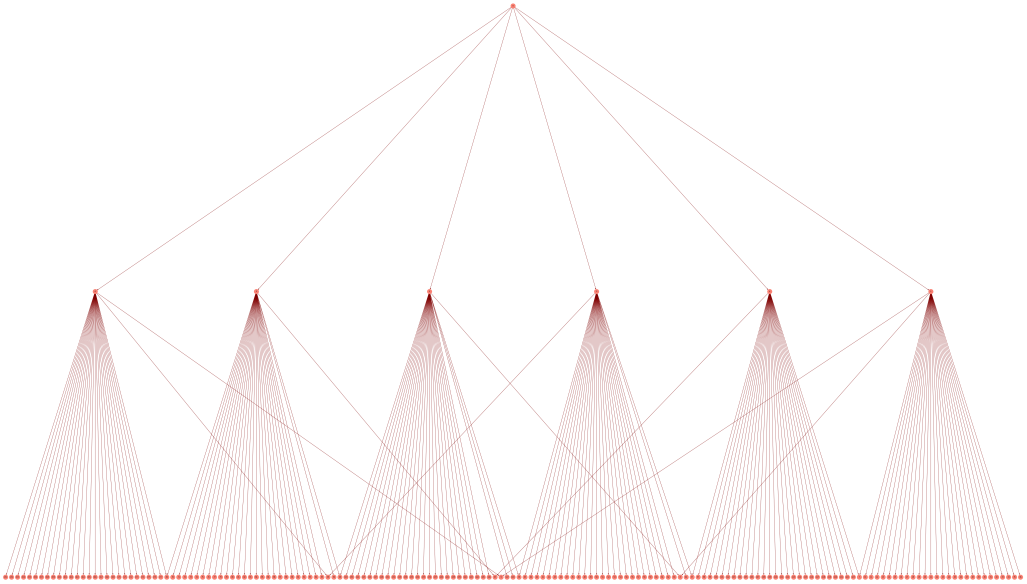

Figure 1: The gamegraph for 3-impartial through level 2 (where 2 cubes have been placed on the board).

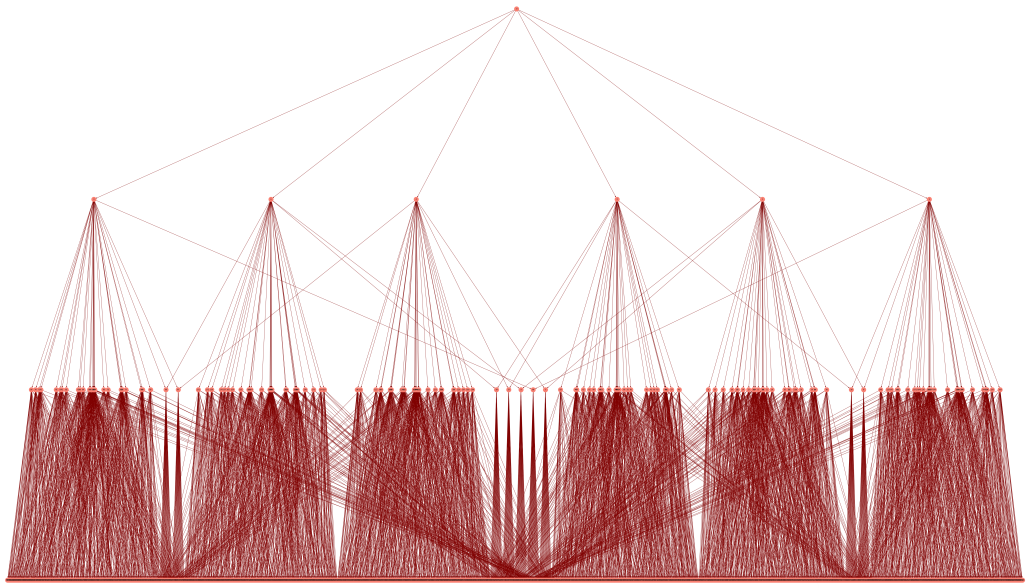

Figure 2: The gamegraph for 3-impartial through level 3 (where 3 cubes have been placed on the board).

Bibliography

[1] Mikhail Baltushkin, Dana C. Ernst, and Nándor Sieben, *Isomorphism theorems for impartial combinatorial games*, 2025.